

Rapport de Stage - Sections Techniques

Projet : Application VR Multiplayer de Réunion Collaborative

Unity 6000.2.14f1 | WebSocket + WebRTC | OpenXR

Section 5 : Environnement technique

5.1 Technologies et outils utilisés

5.1.1 Moteur de jeu : Unity 6000.2.14f1

Le projet repose sur **Unity 6000.2.14f1**, la dernière version LTS (Long Term Support) du moteur de jeu Unity. Cette version offre une stabilité accrue et un support étendu pour les technologies XR (Extended Reality). Unity a été choisi pour plusieurs raisons :

- **Écosystème mature** : Large communauté, documentation extensive et nombreux packages disponibles
- **Support multiplateforme** : Compilation native pour Quest (Android), PCVR (Windows) et Desktop
- **Pipeline de rendu URP** : Universal Render Pipeline optimisé pour la VR avec le single-pass instanced rendering
- **XR Interaction Toolkit** : Framework officiel pour les interactions VR

5.1.2 Architecture réseau

WebSocket (NativeWebSocket)

La communication temps réel entre les clients et le serveur utilise le protocole WebSocket via la bibliothèque `NativeWebSocket`. Ce choix s'explique par :

- **Connexion persistante** : Contrairement à HTTP, WebSocket maintient une connexion bidirectionnelle permanente
- **Faible latence** : Essentiel pour la synchronisation VR à 30Hz
- **Simplicité** : API légère, compatible avec le navigateur et les applications natives

WebRTC (Unity WebRTC 3.0.0)

La communication vocale utilise WebRTC pour établir des connexions peer-to-peer :

- **Audio haute qualité** : Codec Opus avec suppression d'écho et réduction de bruit
- **Topologie mesh** : Chaque participant parle directement aux autres (jusqu'à 8 personnes)
- **NAT traversal** : Support STUN/TURN pour traverser les pare-feux

5.1.3 Serveur backend

Node.js avec ws

Le serveur est développé en Node.js avec la bibliothèque `ws` :

```
Server/
├─ server.js           # Serveur WebSocket principal
├─ filePresentation.js # Conversion PDF → PNG
└─ src/
    ├─ auth.js         # Authentification JWT + bcrypt
    └─ database.js      # Connexion MariaDB
```

Base de données : MariaDB

MariaDB stocke les données utilisateurs. L'architecture impose une règle stricte : **jamais de connexion directe depuis Unity**. Toutes les requêtes passent par le serveur Node.js qui agit comme intermédiaire sécurisé.

5.1.4 Packages Unity essentiels

Package	Version	Rôle
<code>com.unity.xr.openxr</code>	1.16.1	Loader OpenXR pour Quest/PCVR
<code>com.unity.xr.interaction.toolkit</code>	3.2.2	Interactions VR (grab, teleport, pointeur)
<code>com.unity.xr.hands</code>	1.7.2	Suivi des mains (hand tracking)

Package	Version	Rôle
com.endel.nativewebsocket	git	Client WebSocket natif
com.unity.webrtc	3.0.0	Connexions WebRTC pour la voix
com.unity.render-pipelines.universal	17.2.0	Pipeline de rendu URP
com.veriorpies.parrelsync	git	Test multi-instances

5.2 Pourquoi ces choix technologiques

5.2.1 Unity vs Unreal Engine

Le choix d'Unity s'est imposé pour plusieurs raisons :

1. **Expertise de l'équipe** : Maîtrise préexistante de C# et de l'écosystème Unity
2. **Rapidité de prototypage** : L'éditeur visuel et le hot-reload accélèrent le développement
3. **Poids des builds** : Les applications Unity sont plus légères que leurs équivalents Unreal, crucial pour Quest
4. **XR Interaction Toolkit** : Framework mature et bien documenté pour la VR

5.2.2 WebSocket vs alternatives

Solution	Avantages	Inconvénients	Choix
WebSocket	Temps réel, léger, bidirectionnel	Pas de garantie de livraison	Retenu
HTTP polling	Simple, compatible partout	Latence élevée, consommation réseau	Rejeté
gRPC	Performant, typage fort	Complexité, overhead pour petits messages	Rejeté
Socket.io	Auto-reconnexion, rooms	Overhead JavaScript, moins adapté à Unity	Rejeté

WebSocket offre le meilleur compromis entre simplicité et performance pour la synchronisation VR à 30Hz.

5.2.3 WebRTC pour la voix

Les alternatives considérées :

- **Audio via WebSocket** : Trop de latence et consommation serveur excessive
- **Solution propriétaire (Photon Voice)** : Coût par utilisateur, dépendance externe
- **WebRTC** : Standard ouvert, peer-to-peer, codec optimisé

WebRTC permet une communication vocale de haute qualité sans surcharger le serveur central.

5.3 Infrastructure existante avant le stage

Avant mon arrivée, le projet disposait de :

Éléments existants :

- Structure de base Unity avec scènes vides
- Configuration XR minimale (OpenXR loader)
- Aucune logique réseau implémentée

Éléments à développer :

- Architecture réseau complète (WebSocket + WebRTC)
- Système de salles et synchronisation joueurs
- Interactions VR (whiteboard, laser, partage d'écran)
- Interface utilisateur (menus, authentification)
- Système d'enregistrement

L'infrastructure était donc essentiellement vierge, nécessitant une conception from scratch de l'architecture logicielle.

5.4 Environnement de développement

5.4.1 Outils de développement

- **IDE** : Visual Studio 2022 / Rider pour C#
- **Éditeur** : Unity Editor 6000.2.14f1
- **Git** : Gestion de version avec branches (main, develop, feature/*)
- **Test VR** : Quest 2 en mode Link ou standalone
- **Test multi-joueurs** : ParrelSync pour simuler plusieurs clients

5.4.2 Configuration de test

```
# Lancement du serveur de développement
cd Server/ && npm install && npm run dev

# URL du serveur
ws://localhost:8080
```

Le projet supporte un **mode offline** pour tester sans serveur :

```
// Dans l'Inspector de VRNetworkManager
offlineMode = true
offlineAutoCreateRoom = true
offlineRoomType = MeetingRoomA
```

Section 6 : Chemin critique

6.1 Les grandes étapes du projet

Le développement s'est organisé en six phases principales, suivant une approche itérative :

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5	Phase 6
Architecture & Bootstrap	→ Réseau base & Salles	→ Interactions VR	→ Fonctionnalités avancées	→ Optimisation VR	→ Finalisation

Phase 1 : Architecture et Bootstrap (Semaines 1-2)

Objectifs :

- Définir l'architecture globale du projet
- Mettre en place le système de scènes
- Créer les singletons managers

Livrables :

- `BootstrapManager.cs` : Orchestration du chargement de scènes

- Pattern singleton avec DontDestroyOnLoad
- Structure de dossiers (Assets/Scripts/)
- Scènes Bootstrap (persistante) et Meet (additive)

Phase 2 : Réseau de base et système de salles (Semaines 3-5)

Objectifs :

- Implémenter la connexion WebSocket
- Créer le système de salles
- Synchroniser les positions des joueurs

Livrables :

- VRNetworkManager.cs : Gestion WebSocket avec auto-reconnexion
- VRRoomManager.cs : Création/jointure de salles (codes 6 caractères)
- VRGameManager.cs : Spawn des joueurs locaux et distants
- Serveur Node.js (server.js) avec gestion des messages

Phase 3 : Interactions VR (Semaines 6-8)

Objectifs :

- Locomotion VR (téléportation, déplacement continu)
- Mode Desktop alternatif
- Interactions de base (grab, pointer)

Livrables :

- VRPlayerController.cs : Locomotion snap/smooth turn
- DesktopPlayerController.cs : Contrôles WASD + souris
- LaserPointer.cs : Pointeur laser synchronisé
- Configuration des layers XR (Layer 31 pour téléportation)

Phase 4 : Fonctionnalités avancées (Semaines 9-14)

Objectifs :

- Tableau blanc collaboratif
- Communication vocale WebRTC
- Partage d'écran et de fichiers

Livrables :

- Système whiteboard 3 couches
- VoiceChatManager.cs et sous-composants WebRTC
- ScreenShareManager.cs : Capture et diffusion d'écran
- FileShareManager.cs : Upload/download de fichiers

Phase 5 : Optimisation VR (Semaines 15-18)

Objectifs :

- Système d'enregistrement sans motion sickness
- Optimisation des performances réseau
- Interface utilisateur VR

Livrables :

- Pipeline d'enregistrement async (3 threads)
- Rate limiting (60 msg/s) et message caching
- VRMenuUI.cs : Menu in-game avec pagination

Phase 6 : Finalisation (Semaines 19-20)

Objectifs :

- Intégration de l'authentification
- Tests multi-utilisateurs
- Documentation et polish

Livrables :

- AuthManager.cs et AuthUI.cs
- LaunchLoadingScreen.cs : Écran de chargement progressif
- Documentation [CLAUDE.md](#)

6.2 Étude des solutions envisagées

6.2.1 Synchronisation réseau : 30Hz vs 60Hz

Problème : Quelle fréquence de synchronisation pour les positions VR ?

Fréquence	Bande passante	Fluidité	Charge serveur
60 Hz	~120 KB/s/joueur	Excellente	Élevée
30 Hz	~60 KB/s/joueur	Bonne	Modérée
15 Hz	~30 KB/s/joueur	Acceptable	Faible

Solution retenue : 30 Hz avec interpolation

L'interpolation sur 15 frames compense la fréquence réduite. Un seuil de mouvement (0.01m / 1°) évite d'envoyer des messages inutiles quand le joueur est immobile.

6.2.2 Architecture vocale : Mesh vs SFU

Problème : Comment connecter les participants pour la voix ?

Architecture	Complexité serveur	Latence	Scalabilité
Mesh (P2P)	Nulle	Minimale	~8 personnes
SFU (Selective Forwarding Unit)	Élevée	Faible	50+ personnes
MCU (Mixing)	Très élevée	Moyenne	100+ personnes

Solution retenue : Mesh P2P

Pour des réunions de 2-8 personnes, le mesh P2P offre la latence la plus faible sans infrastructure serveur supplémentaire. Le client avec l'ID le plus petit initie les connexions pour éviter les connexions dupliquées.

6.2.3 Tableau blanc : Couches séparées vs surface unique

Problème : Comment gérer le dessin local et réseau ?

Approche initiale (rejetée) : Surface unique où chaque trait est dessiné localement puis envoyé au réseau, créant des doublons visuels.

Solution retenue : Architecture 3 couches

Couche 1: Whiteboard.cs → Fond blanc + mode présentation
 Couche 2: WhiteboardDrawingSurface.cs → Dessins réseau (ne dessine PAS localement)
 Couche 3: WhiteboardMarker.cs → Dessin local uniquement

Cette séparation évite les doublons et permet le mode présentation (partage d'écran sur la couche 1).

6.3 Difficultés rencontrées et solutions

6.3.1 Motion sickness lors de l'enregistrement

Problème : La capture GPU bloquante causait des micro-freezes, provoquant le mal des transports en VR.

Investigation :

- `Graphics.CopyTexture()` bloquait le thread principal (~16ms de gel)
- En VR, tout freeze > 11ms cause des nausées

Solution : Pipeline asynchrone à 3 étages

```
// Thread principal (0.1ms)
AsyncGPUReadback.Request(texture, callback);
```

```
// Thread d'encodage (background)
ConcurrentQueue<FrameData> encodeQueue;
```

```
// Thread d'écriture (background)
ConcurrentQueue<byte[]> writeQueue;
```

`AsyncGPUReadback` retourne immédiatement, le traitement se fait en arrière-plan.

6.3.2 Conflits XR Simulator et tracking Quest

Problème : Le simulateur XR d'Unity interférait avec le tracking natif Quest.

Symptômes :

- Position de la caméra aléatoire
- Mains figées ou tremblantes
- Perte de tracking intermittente

Solution :

```
private void DisableXRSimulatorInVRMode()
{
    if (XRGeneralSettings.Instance?.Manager?.activeLoader != null)
    {
        // Désactiver le simulateur quand un vrai casque est connecté
        var simulator = FindObjectOfType<XRDeviceSimulator>();
        if (simulator != null) simulator.enabled = false;
    }
}
```

6.3.3 Synchronisation des late joiners

Problème : Les joueurs rejoignant une session en cours ne voyaient pas l'état actuel (dessins whiteboard, fichiers partagés, etc.).

Solution : Pattern Request/State

```

Nouveau joueur rejoint
    ↓
Envoie "whiteboard-request" + "file-share-request"
    ↓
Serveur ou host répond avec "-state"
    ↓
Client applique l'état reçu

```

Chaque système (whiteboard, screen share, file share) implémente ce pattern.

6.3.4 Sérialisation JSON avec objets imbriqués

Problème : JsonUtility d'Unity ne supporte pas les objets imbriqués ou les dictionnaires.

Exemple problématique :

```
// NE FONCTIONNE PAS avec JsonUtility
public class Message {
    public Dictionary<string, object> data; // Erreur
    public NestedClass nested; // Erreur si NestedClass non [Serializable]
}
```

Solution : Aplatir les structures et utiliser des classes [Serializable]

```

[Serializable]
public class NetworkMessage {
    public string type;
    public string senderId;
    public string data; // Données sérialisées en string, pas en objet
}

// Sérialisation en deux étapes
var payload = JsonUtility.ToJson(myData);
var message = new NetworkMessage { type = "room-join", data = payload };
var json = JsonUtility.ToJson(message);

```

6.3.5 Layer Teleport vs Layer Grab

Problème : Les objets grabbables déclenchaient accidentellement la téléportation.

Cause : Le raycast de téléportation touchait les mêmes colliders que le grab.

Solution : Séparation stricte des layers

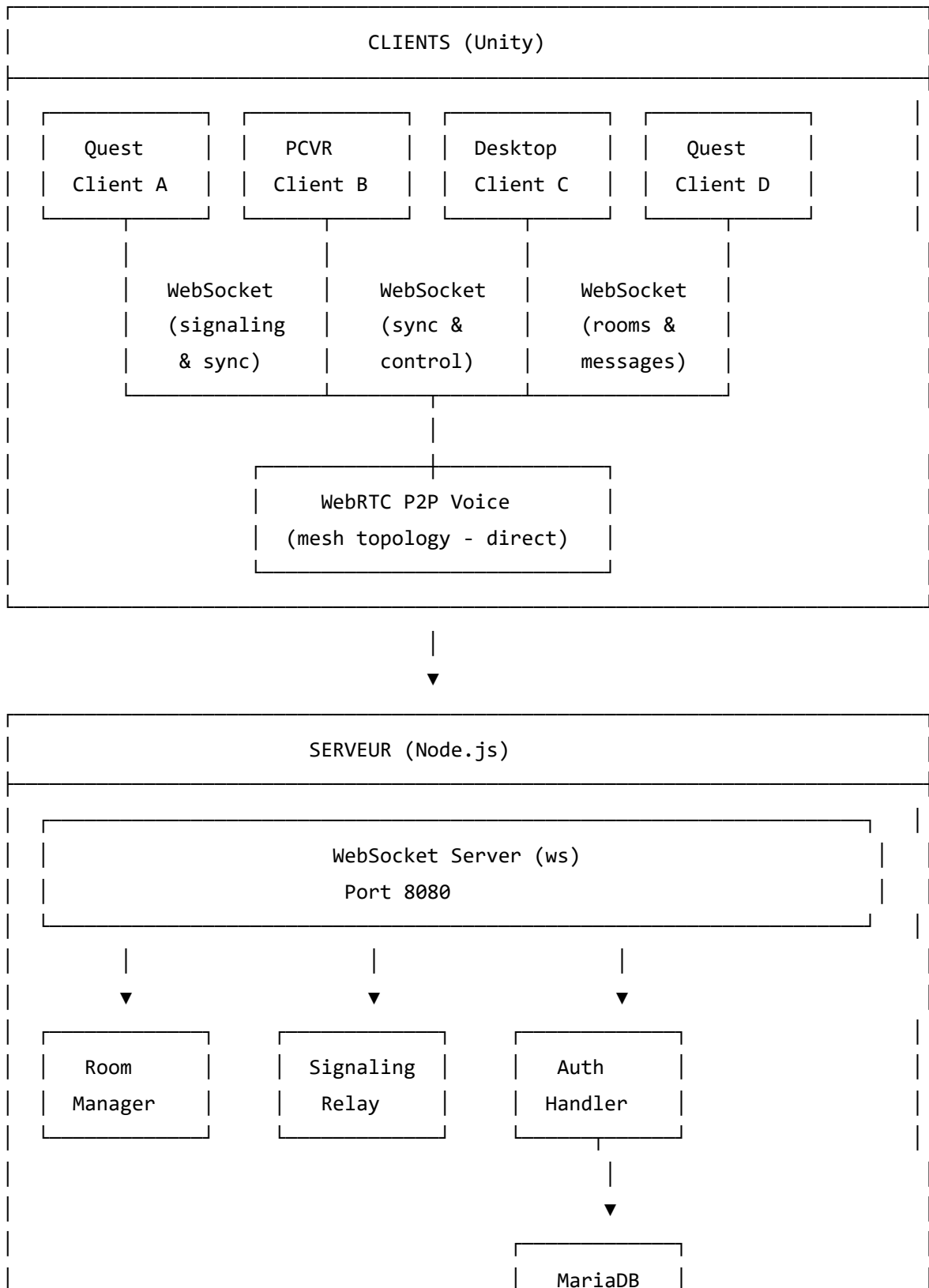
Layer 31: Teleport	→ Uniquement les surfaces de téléportation
Layer Default: Grab	→ Objets interactifs (ne doit PAS inclure Layer 31)

Configuration dans les Interactors XR :

- Teleport Interactor : Layer Mask = Layer 31 uniquement
- Direct/Ray Interactor : Layer Mask = Tout SAUF Layer 31

Section 7 : Architecture de la solution

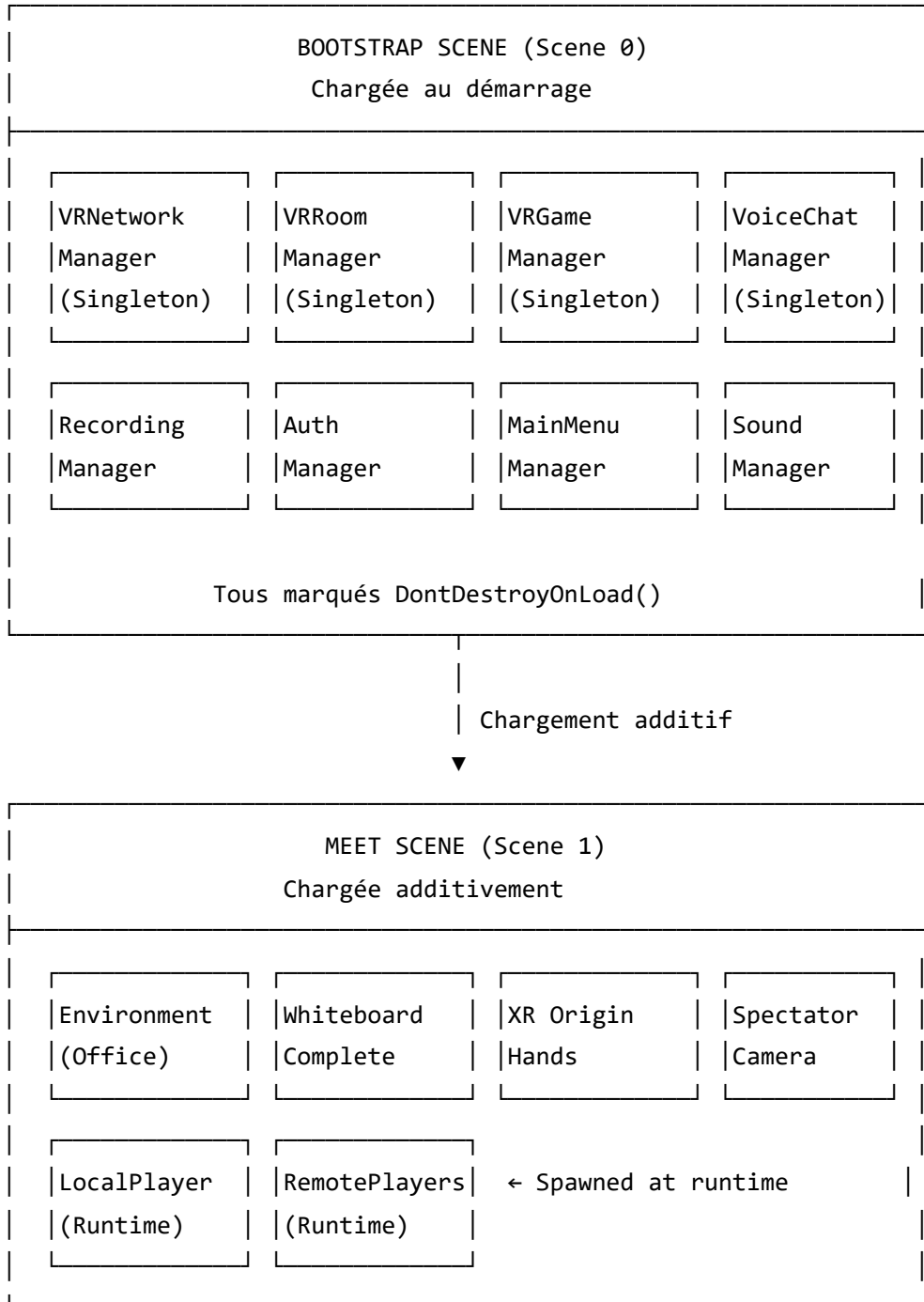
7.1 Schéma d'ensemble simplifié



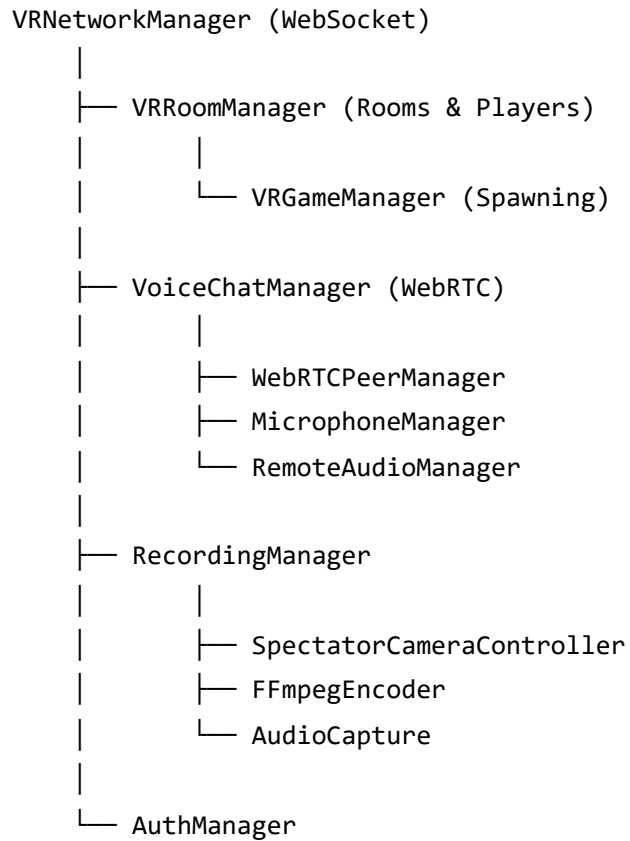
(users)

7.2 Architecture côté client (Unity)

7.2.1 Organisation des scènes



7.2.2 Hiérarchie des managers



7.3 Communication inter-composants

7.3.1 Pattern événementiel

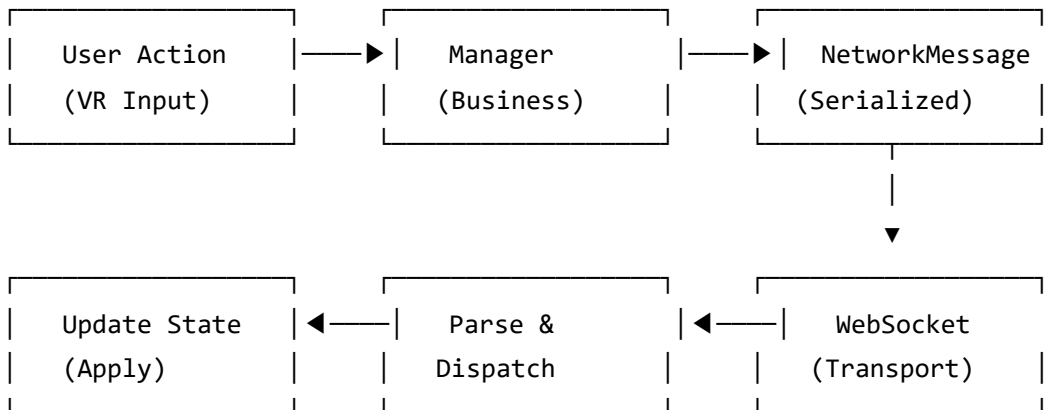
Les composants communiquent via des événements C# :

```
// Définition dans VRNetworkManager
public static event Action OnConnected;
public static event Action<string> OnPeerConnected;
public static event Action<NetworkMessage> OnMessageReceived;

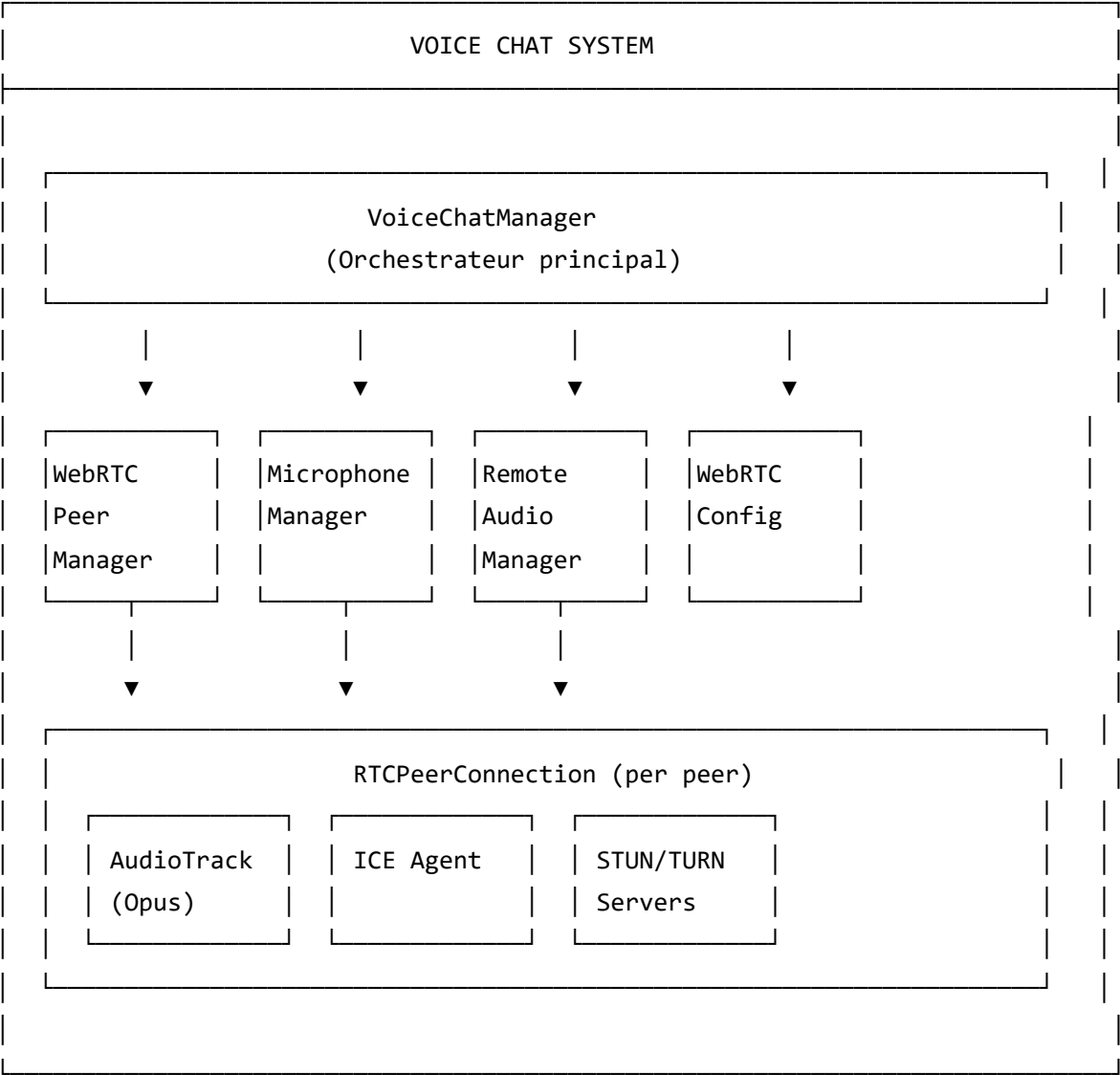
// Souscription dans VRRoomManager
void OnEnable() {
    VRNetworkManager.OnConnected += HandleConnected;
    VRNetworkManager.OnMessageReceived += HandleMessage;
}

void OnDisable() {
    VRNetworkManager.OnConnected -= HandleConnected;
    VRNetworkManager.OnMessageReceived -= HandleMessage;
}
```

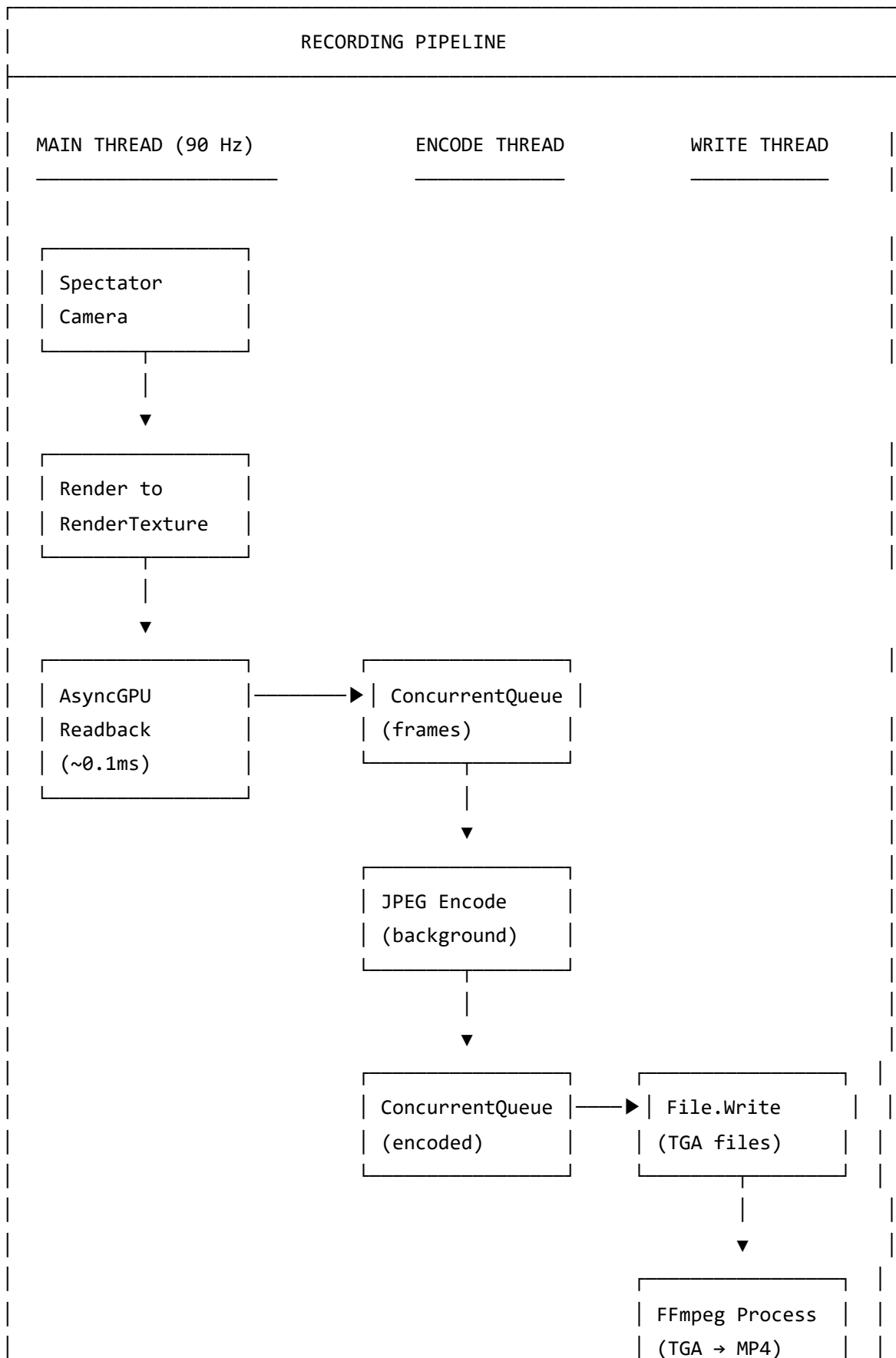
7.3.2 Flux de données réseau



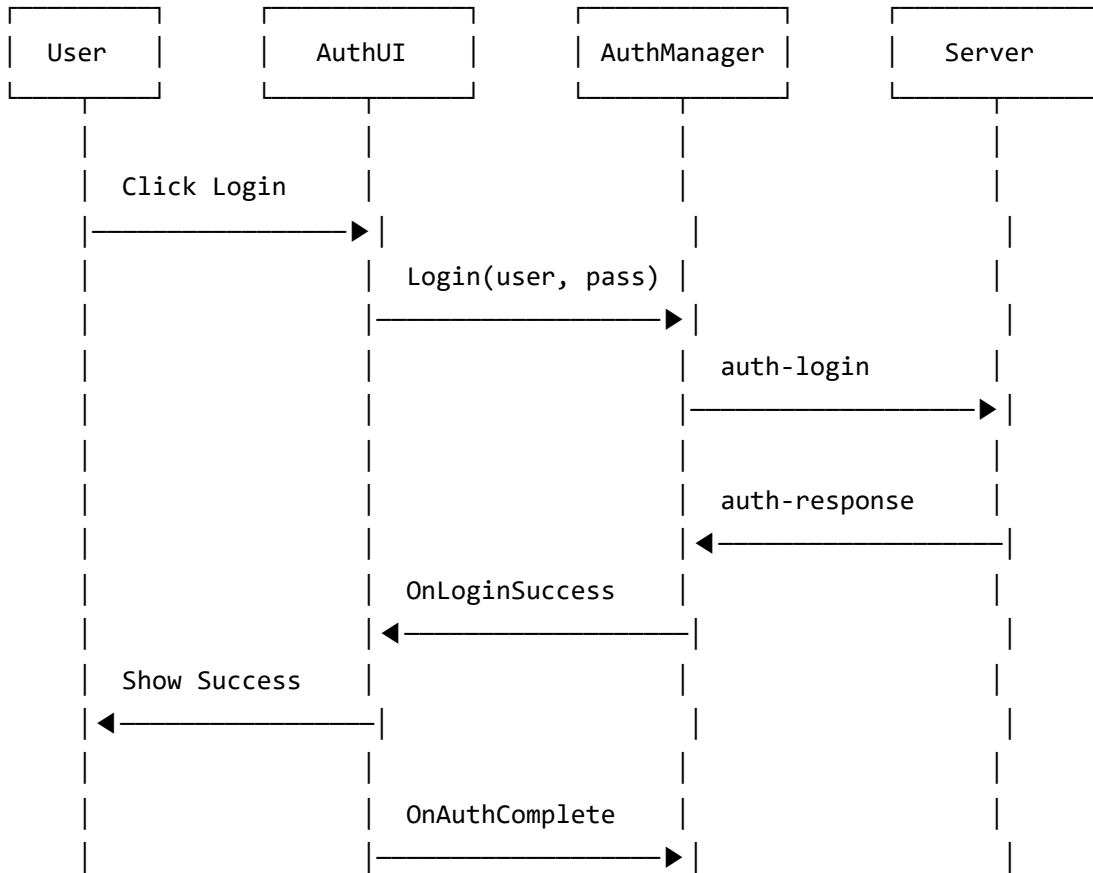
7.4 Architecture du système vocal



7.5 Architecture du système d'enregistrement



7.6 Flux d'authentification



Section 8 : Éléments importants du développement

8.1 Système de connexion réseau

8.1.1 Gestion WebSocket avec reconnexion automatique

Le `VRNetworkManager` implémente une connexion WebSocket robuste avec :

Reconnexion exponentielle :

```
private float _reconnectDelay = 1f;
private const float MAX_RECONNECT_DELAY = 30f;

private IEnumerator ReconnectCoroutine()
{
    while (!_isConnected && _shouldReconnect)
    {
        yield return new WaitForSeconds(_reconnectDelay);
        Connect();
        _reconnectDelay = Mathf.Min(_reconnectDelay * 2, MAX_RECONNECT_DELAY);
    }
}
```

Rate limiting :

- Maximum 60 messages/seconde
- Burst allowance de 10 messages
- Protection contre le flooding accidentel lors de mouvements rapides

8.1.2 Protocole de messages

Chaque message suit une structure simple pour compatibilité avec `JsonUtility` :

```
[Serializable]
public class NetworkMessage
{
    public string type;        // Type de message (ex: "room-join")
    public string senderId;    // ID du client émetteur
    public string data;        // Payload JSON sérialisé en string
}
```

Catégories de messages :

Catégorie	Messages	Fréquence
Connexion	welcome, peer-connected, peer-disconnected	Événementiel
Salles	room-join, room-leave, room-list, room-teleport	Événementiel
VR Sync	vr-position	30 Hz

Catégorie	Messages	Fréquence
Voix	webrtc-offer, webrtc-answer, webrtc-ice-candidate	Événementiel
Whiteboard	whiteboard-batch, whiteboard-clear	30 Hz (batches)
Partage	screen-share-frame, file-share-*	3-5 Hz

8.2 Système de salles (Rooms)

8.2.1 Types de salles

Le projet définit trois types de salles :

```
public enum RoomType
{
    Lobby,           // Espace d'attente
    MeetingRoomA,    // Salle de réunion principale
    MeetingRoomB     // Salle de réunion secondaire
}
```

8.2.2 Création et jointure

Codes de salle : 6 caractères alphanumériques générés aléatoirement

```
private string GenerateRoomCode()
{
    const string chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    var random = new System.Random();
    return new string(Enumerable.Repeat(chars, 6)
        .Select(s => s[random.Next(s.Length)]).ToArray());
}
```

Autorité host : Le créateur de la salle a l'autorité sur certaines actions (démarrage d'enregistrement, kick, etc.).

8.2.3 Synchronisation des late joiners

Quand un joueur rejoint une session en cours, il doit recevoir l'état actuel :

1. Client envoie "whiteboard-request"
2. Host répond avec "whiteboard-state" (dessins existants)
3. Client envoie "screen-share-request"
4. Host répond avec "screen-share-state" (si partage actif)
5. Client envoie "file-share-request"
6. Host répond avec "file-share-state" (fichiers partagés)

8.3 Locomotion VR

8.3.1 Modes de déplacement

Snap Turn (par défaut) :

- Rotation instantanée de 45° (configurable)
- Moins de motion sickness
- Activation : Thumbstick droit gauche/droite

Smooth Turn :

- Rotation continue à 90°/seconde
- Plus immersif mais peut causer des nausées
- Option dans les paramètres

Déplacement continu :

- Direction basée sur l'orientation de la tête
- Vitesse : 2 m/s (3 m/s en sprint)
- Activation : Thumbstick gauche

8.3.2 Téléportation

La téléportation utilise le Layer 31 dédié :

```
// Configuration du Teleport Interactor
[SerializeField] private LayerMask teleportLayerMask = 1 << 31;

// Seules les surfaces sur ce layer peuvent être ciblées
```

Processus :

1. Le joueur pointe avec le contrôleur
2. Un arc de téléportation s'affiche (XR Interactor)
3. Si la destination est valide (Layer 31), le point devient vert
4. Relâcher active la téléportation instantanée

8.4 Système de tableau blanc (Whiteboard)

8.4.1 Architecture 3 couches

Cette architecture résout le problème des doublons de traits :

Couche 1 - `Whiteboard.cs` :

- Texture de fond blanche (2048x2048)
- Gère le mode présentation (affiche le partage d'écran)
- Shader : Sprites/Default

Couche 2 - `WhiteboardDrawingSurface.cs` :

- Texture transparente overlay
- Reçoit uniquement les dessins réseau
- **Important** : Ne dessine PAS les traits locaux

Couche 3 - `WhiteboardMarker.cs` / `DesktopWhiteboardDrawer.cs` :

- Dessin local uniquement
- Envoie les points au réseau par batches (33ms)
- Chaque point inclut position UV, couleur, épaisseur

8.4.2 Synchronisation des dessins

```
[Serializable]
public class WhiteboardBatch
{
    public string whiteboardId;
    public WhiteboardPoint[] points;
    public float brushSize;
    public Color color;
}

[Serializable]
public class WhiteboardPoint
{
    public float u; // Coordonnée UV X
    public float v; // Coordonnée UV Y
}
```

Les points sont accumulés localement puis envoyés toutes les 33ms pour réduire le trafic réseau.

8.5 Communication vocale WebRTC

8.5.1 Établissement des connexions

La topologie mesh signifie que chaque participant se connecte directement aux autres :

Pour 4 participants (A, B, C, D) :

A ↔ B

A ↔ C

A ↔ D

B ↔ C

B ↔ D

C ↔ D

Total : 6 connexions P2P

Règle d'initiation : Le client avec l'ID le plus petit initie la connexion pour éviter les doublons.

8.5.2 Configuration STUN/TURN

```
private RTCCConfiguration CreateConfiguration()
{
    var config = new RTCCConfiguration
    {
        iceServers = new[]
        {
            new RTCIceServer { urls = new[] { "stun:stun.l.google.com:19302" } },
            new RTCIceServer
            {
                urls = new[] { "turn:your-turn-server.com:3478" },
                username = "user",
                credential = "pass"
            }
        }
    };
    return config;
}
```

8.5.3 Audio spatial 3D

Le son des participants est spatialisé en 3D :

```
// Attaché à la tête du joueur distant
private void SetupSpatialAudio(AudioSource source)
{
    source.spatialBlend = 1f;           // 100% 3D
    source.minDistance = 1f;           // Plein volume à 1m
    source.maxDistance = 20f;          // Inaudible au-delà
    source.rolloffMode = AudioRolloffMode.Linear;
}
```

8.6 Partage d'écran

8.6.1 Capture de fenêtre (Windows)

Le `WindowCapture.cs` utilise les API Windows natives :


```
[DllImport("user32.dll")]
private static extern bool EnumWindows(EnumWindowsProc lpEnumFunc, IntPtr lParam);

[DllImport("user32.dll")]
private static extern IntPtr GetDC(IntPtr hWnd);
```

Processus de capture :

1. Énumération des fenêtres visibles
2. L'utilisateur sélectionne une fenêtre
3. Capture du contenu à 3-5 fps
4. Redimensionnement à 854x480
5. Compression JPEG (50%)
6. Envoi via WebSocket

8.6.2 Affichage sur le tableau blanc

Quand le partage est actif, la Couche 1 du whiteboard passe en mode présentation :

```
public void SetPresentationMode(Texture2D screenTexture)
{
    _isPresentationMode = true;
    _backgroundRenderer.material.mainTexture = screenTexture;
}
```

Les dessins restent possibles par-dessus (Couche 2 et 3).

8.7 Système d'enregistrement VR-optimisé

8.7.1 Problématique VR

En VR, la fluidité est critique. Un freeze de plus de 11ms provoque :

- Désynchronisation entre mouvements réels et affichage
- Nausées et mal des transports
- Expérience utilisateur dégradée

Les méthodes classiques de capture (`Graphics.CopyTexture` , `ReadPixels`) bloquent le thread principal pendant 10-20ms.

8.7.2 Solution : Pipeline asynchrone

Étape 1 - Capture non-bloquante :

```
AsyncGPUReadback.Request(renderTexture, 0, TextureFormat.RGB24, OnReadbackComplete);  
// Retourne immédiatement (~0.1ms)
```

Étape 2 - Encodage en arrière-plan :

```
private ConcurrentQueue<NativeArray<byte>> _encodeQueue;  
  
private void EncodeThread()  
{  
    while (_isRecording)  
    {  
        if (_encodeQueue.TryDequeue(out var pixels))  
        {  
            var tga = EncodeToTGA(pixels);  
            _writeQueue.Enqueue(tga);  
        }  
    }  
}
```

Étape 3 - Écriture I/O séparée :

```
private void WriteThread()  
{  
    while (_isRecording)  
    {  
        if (_writeQueue.TryDequeue(out var data))  
        {  
            File.WriteAllBytes(GetNextFramePath(), data);  
        }  
    }  
}
```

8.7.3 Post-traitement FFmpeg

À la fin de l'enregistrement, FFmpeg combine les frames :

```
ffmpeg -framerate 30 -i frames/%04d.tga -i audio.wav \
-c:v libx264 -preset fast -crf 23 \
-c:a aac -b:a 128k \
output.mp4
```

8.8 Interface utilisateur VR

8.8.1 Adaptation des Canvas

Les Canvas Unity sont conçus pour les écrans 2D. En VR, ils doivent être transformés :

```
public class VRCanvasAdapter : MonoBehaviour
{
    void Awake()
    {
        var canvas = GetComponent<Canvas>();
        canvas.renderMode = RenderMode.WorldSpace;

        // Positionnement devant le joueur
        transform.localScale = Vector3.one * 0.001f; // 1 unité = 1mm
        transform.position = Camera.main.transform.position +
            Camera.main.transform.forward * 2f;
    }
}
```

8.8.2 Menu VR avec pagination

Le VRMenuUI.cs implémente un menu flottant avec :

- **Pagination** : Navigation entre les pages d'options
- **Sidebar** : Accès rapide aux sections principales
- **Follow behavior** : Le menu suit le regard du joueur
- **Interaction** : Pointeur laser pour sélectionner

8.9 Gestion des avatars

8.9.1 Personnalisation

Les joueurs peuvent personnaliser :

- **Couleur** : 8 couleurs prédéfinies
- **Nom** : Affiché au-dessus de la tête

```
[Serializable]
public class AvatarData
{
    public string playerId;
    public string displayName;
    public int colorIndex;
}
```

8.9.2 Synchronisation réseau

Les changements d'avatar sont envoyés via `avatar-update` :

```
public void UpdateAvatar(int colorIndex, string name)
{
    var data = new AvatarData
    {
        playerId = _localPlayerId,
        colorIndex = colorIndex,
        displayName = name
    };
    VRNetworkManager.Instance.SendMessage("avatar-update", JsonUtility.ToJson(data));
}
```

8.10 Optimisations de performance

8.10.1 Caching des messages

Pour éviter les allocations GC lors de la synchronisation 30Hz :

```
// Mauvais : nouvelle allocation à chaque frame
void Update()
{
    var msg = new NetworkMessage { type = "vr-position", ... }; // GC!
}

// Bon : réutilisation d'un objet caché
private readonly NetworkMessage _cachedMessage = new();

void Update()
{
    _cachedMessage.type = "vr-position";
    _cachedMessage.data = ...;
    // Pas d'allocation
}
```

8.10.2 Seuils de mouvement

Les mises à jour de position ne sont envoyées que si le joueur a réellement bougé :

```
private const float POSITION_THRESHOLD = 0.01f; // 1cm
private const float ROTATION_THRESHOLD = 1f;    // 1 degré

bool ShouldSendUpdate()
{
    return Vector3.Distance(_lastPosition, transform.position) > POSITION_THRESHOLD
        || Quaternion.Angle(_lastRotation, transform.rotation) > ROTATION_THRESHOLD;
}
```

8.10.3 Interpolation des mouvements distants

Les positions reçues à 30Hz sont interpolées pour un rendu à 90Hz :

```
private void Update()
{
    _interpolationProgress += Time.deltaTime * 30f; // 30 Hz target
    _interpolationProgress = Mathf.Clamp01(_interpolationProgress);

    transform.position = Vector3.Lerp(_lastReceivedPos, _targetPos, _interpolationProgress);
    transform.rotation = Quaternion.Slerp(_lastReceivedRot, _targetRot, _interpolationProgress);
}
```

Annexes suggérées

A. Diagramme de classes complet

À générer avec un outil UML à partir des scripts du projet.

B. Captures d'écran

- Interface du menu principal
- Vue VR de la salle de réunion
- Tableau blanc en mode dessin
- Menu VR flottant

C. Protocole réseau complet

Liste exhaustive de tous les types de messages et leur format JSON.

D. Configuration serveur

Instructions de déploiement et variables d'environnement requises.